Bash-script Task-2

1. Create one Bash script to check if a directory is available or not.

- Create the script using vi: vi check_directory.bash
- Press i to enter Insert mode.
- Write the following code:#!/bin/bash
-
- DIR="/kamaal/scripts"
-
- if [-d "$DIR"]; then
- echo "Directory already exists."
- else
- mkdir -p "$DIR"
- echo "Directory created successfully."
- fi
- Press Esc, type :wq, and press Enter to save and exit.
- Give execution permission: chmod 755 check_directory.bash
- Run the script: ./check_directory.bash
- Verify the directory: ls -ld /kamaal/scripts

1.
```
[[root@ip-172-31-31-201 scripts]# vi check_directory.bash
[[root@ip-172-31-31-201 scripts]# ./check_directory.bash
directory exist
[[root@ip-172-31-31-201 scripts]# cat check_directory.bash
#!/bin/bash

dir="/kamaal/scripts"

if [ -d "$dir" ]; then
        echo "directory exist"
else
        echo "directory doesnot exist"
fi
[[root@ip-172-31-31-201 scripts]# chmod 755 check_directory.bash
[[root@ip-172-31-31-201 scripts]# ./check_directory.bash
directory exist
[root@ip-172-31-31-201 scripts]#
```

2.Create a Bash script which will create multiple files.

vi create_files.bash

- Press i to enter Insert mode.
- Write the following code:
  #!/bin/bash
-
- DIR="/kamaal/scripts"
-
- mkdir -p "$DIR"

-
- touch "$DIR/file1.txt" "$DIR/file2.txt" "$DIR/file3.txt" "$DIR/file4.txt" "$DIR/file5.txt"
-
- echo "Multiple files created successfully."
- Press Esc, type :wq, and press Enter to save and exit.
- Give execution permission: chmod 755 create_files.bash
- Run the script: ./create_files.bash
- Verify the files: ls -l /kamaal/scripts

1.
```
[[root@ip-172-31-31-201 scripts]# vi create_files.bash
[[root@ip-172-31-31-201 scripts]# cat create_files.bash
#!/bin/bash

mkdir -p /kamaal/scripts
cd /kamaal/scripts || exit 1
touch sharaf.txt kamaal.txt techie.txt
echo "multiple files created successfully"

[[root@ip-172-31-31-201 scripts]# chmod 755 create_files.bash
[[root@ip-172-31-31-201 scripts]# vi create_files.bash
[[root@ip-172-31-31-201 scripts]# ./create_files.bash
multiple files created successfully
[[root@ip-172-31-31-201 scripts]# ls -la
total 8
drwxr-xr-x. 2 root root 113 Oct 21 06:41 .
drwxr-xr-x. 3 root root  21 Oct 21 06:18 ..
-rwxr-xr-x. 1 root root 124 Oct 21 06:29 check_directory.bash
-rwxr-xr-x. 1 root root 150 Oct 21 06:41 create_files.bash
-rw-r--r--. 1 root root   0 Oct 21 06:41 kamaal.txt
-rw-r--r--. 1 root root   0 Oct 21 06:41 sharaf.txt
-rw-r--r--. 1 root root   0 Oct 21 06:41 techie.txt
[root@ip-172-31-31-201 scripts]#
```

3.Create a Bash script to take backup of a directory.
vi backup_directory.bash

- Press i to enter Insert mode.

- Write the following code:
  #!/bin/bash
-
- SOURCE="/kamaal/scripts"
- BACKUP="/kamaal/scripts_backup_$(date +%Y%m%d_%H%M%S)"
-
- if [-d "$SOURCE"]; then
- cp -r "$SOURCE" "$BACKUP"
- echo "Directory backup completed successfully."
- echo "Backup location: $BACKUP"

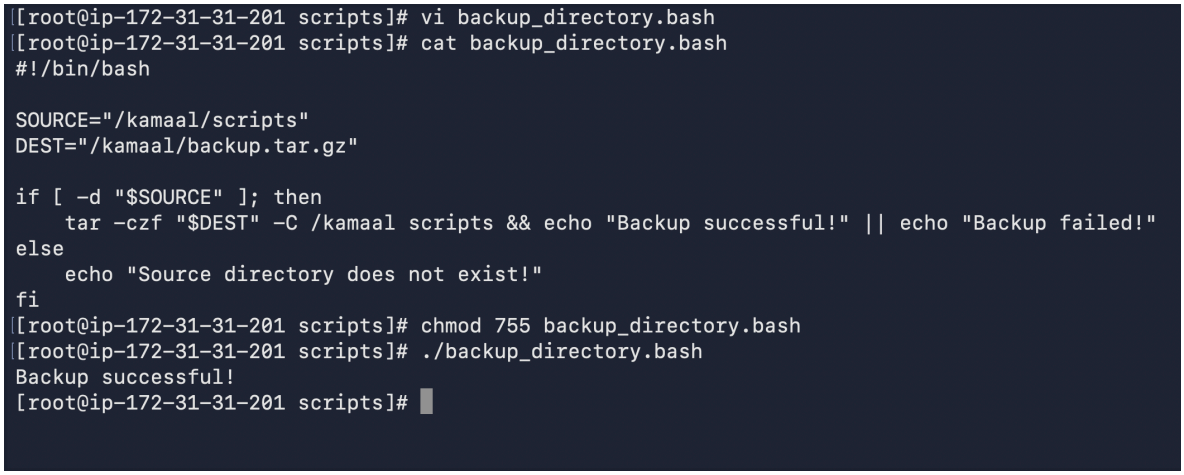
- else
- 	echo "Source directory does not exist."
- fi

- Press Esc, type :wq, and press Enter to save and exit.

- Give execution permission: chmod 755 backup_directory.bash

- Run the script: ./backup_directory.bash

- Verify the backup: ls -ld /kamaal/scripts_backup_*

1. 

4.Create a Bash script to install **nginx** in an EC2 server.
vi install_nginx.bash

- Press i to enter Insert mode.
- Write the following code:
  #!/bin/bash
-
- echo "Installing Nginx..."
-
- sudo dnf install nginx -y
-
- sudo systemctl enable nginx
- sudo systemctl start nginx
-
- if systemctl is-active --quiet nginx; then
- 	echo "Nginx installed and running successfully."

- else
- echo "Nginx installation or startup failed."
- fi
- Press Esc, type :wq, and press Enter to save and exit.
- Give execution permission: chmod 755 install_nginx.bash
- Run the script: ./install_nginx.bash
- Verify the Nginx service: sudo systemctl status nginx
- Test Nginx locally: curl -I http://localhost

1.
```
[root@ip-172-31-31-201 scripts]# vi install_nginx.bash
[root@ip-172-31-31-201 scripts]# cat install_nginx.bash

#!/bin/bash

echo "Checking Nginx installation..."

if ! rpm -q nginx >/dev/null 2>&1; then
    sudo dnf install nginx -y
    if [ $? -ne 0 ]; then
        echo "Nginx installation failed!"
        exit 1
    fi
else
    echo "Nginx is already installed."
fi

echo "Checking Nginx service..."

sudo systemctl enable nginx
sudo systemctl start nginx

if systemctl is-active --quiet nginx; then
    echo "Nginx is running."
else
    echo "Nginx is not running."
    sudo systemctl status nginx --no-pager
    exit 1
fi

if systemctl is-enabled --quiet nginx; then
    echo "Nginx is enabled at boot."
else
    echo "Nginx is not enabled at boot."
```

```
else
    echo "Nginx is not enabled at boot."
    exit 1
fi

echo "Checking HTTP response..."

if curl -fsS http://localhost/ >/dev/null; then
    echo "Nginx HTTP test successful."
else
    echo "Nginx HTTP test failed."
    exit 1
fi

echo "Nginx installation and checks completed successfully!"
```

```
[root@ip-172-31-31-201 scripts]# chmod 755 install_nginx.bash
[root@ip-172-31-31-201 scripts]# ./install_nginx.bash
Checking Nginx installation...
Amazon Linux 2023 Kernel Livepatch repository                   849 kB/s |  89 kB     00:00
Dependencies resolved.
================================================================================================================
 Package                    Architecture       Version                        Repository               Size
================================================================================================================
Installing:
 nginx                      x86_64             1:1.30.5-1.amzn2023.0.1        amazonlinux              34 k
Installing dependencies:
 generic-logos-httpd        noarch             18.0.0-12.amzn2023.0.3         amazonlinux              19 k
 gperftools-libs            x86_64             2.9.1-1.amzn2023.0.3           amazonlinux             308 k
 libunwind                  x86_64             1.4.0-5.amzn2023.0.3           amazonlinux              66 k
 nginx-core                 x86_64             1:1.30.5-1.amzn2023.0.1        amazonlinux             711 k
 nginx-filesystem           noarch             1:1.30.5-1.amzn2023.0.1        amazonlinux              10 k
 nginx-mimetypes            noarch             2.1.49-3.amzn2023.0.3          amazonlinux              21 k

Transaction Summary
================================================================================================================
Install  7 Packages

Total download size: 1.1 M
Installed size: 3.8 M
Downloading Packages:
(1/7): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch.rpm                   640 kB/s |  19 kB     00:00
(2/7): gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64.rpm                         9.0 MB/s | 308 kB     00:00
(3/7): libunwind-1.4.0-5.amzn2023.0.3.x86_64.rpm                               1.7 MB/s |  66 kB     00:00
(4/7): nginx-1.30.5-1.amzn2023.0.1.x86_64.rpm                                  1.4 MB/s |  34 kB     00:00
(5/7): nginx-filesystem-1.30.5-1.amzn2023.0.1.noarch.rpm                       410 kB/s |  10 kB     00:00
(6/7): nginx-core-1.30.5-1.amzn2023.0.1.x86_64.rpm                              17 MB/s | 711 kB     00:00
(7/7): nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch.rpm                        681 kB/s |  21 kB     00:00
----------------------------------------------------------------------------------------------------------------
```

```
Total                                                           7.8 MB/s | 1.1 MB     00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                                        1/1
  Running scriptlet: nginx-filesystem-1:1.30.5-1.amzn2023.0.1.noarch                       1/7
  Installing       : nginx-filesystem-1:1.30.5-1.amzn2023.0.1.noarch                       1/7
  Installing       : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch                          2/7
  Installing       : libunwind-1.4.0-5.amzn2023.0.3.x86_64                                 3/7
  Installing       : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64                           4/7
  Installing       : nginx-core-1:1.30.5-1.amzn2023.0.1.x86_64                             5/7
  Installing       : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch                     6/7
  Installing       : nginx-1:1.30.5-1.amzn2023.0.1.x86_64                                  7/7
  Running scriptlet: nginx-1:1.30.5-1.amzn2023.0.1.x86_64                                  7/7
  Verifying        : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch                     1/7
  Verifying        : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64                           2/7
  Verifying        : libunwind-1.4.0-5.amzn2023.0.3.x86_64                                 3/7
  Verifying        : nginx-1:1.30.5-1.amzn2023.0.1.x86_64                                  4/7
  Verifying        : nginx-core-1:1.30.5-1.amzn2023.0.1.x86_64                             5/7
  Verifying        : nginx-filesystem-1:1.30.5-1.amzn2023.0.1.noarch                       6/7
  Verifying        : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch                          7/7

Installed:
  generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch           gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64
  libunwind-1.4.0-5.amzn2023.0.3.x86_64                       nginx-1:1.30.5-1.amzn2023.0.1.x86_64
  nginx-core-1:1.30.5-1.amzn2023.0.1.x86_64                   nginx-filesystem-1:1.30.5-1.amzn2023.0.1.noarch
  nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch

Complete!
Checking Nginx service...
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service.
Nginx is running.
```

5.Create a Bash script to install **Apache Tomcat** in an EC2 server.

vi install_tomcat.bash

- Press i to enter Insert mode.

- Write the following code. This script is intended for Amazon Linux 2023 and installs Java 17 and Apache Tomcat 10.
  #!/bin/bash
-
- echo "Installing Java..."
- sudo dnf install java-17-amazon-corretto -y
-
- echo "Creating Tomcat user..."
- sudo useradd -r -m -U -d /opt/tomcat -s /bin/false tomcat 2>/dev/null || true
-
- echo "Downloading Apache Tomcat..."
- cd /tmp || exit 1
- curl -fLO https://archive.apache.org/dist/tomcat/tomcat-10/v10.1.49/bin/apache-tomcat-10.1.49.tar.gz
-
- echo "Installing Tomcat..."
- sudo mkdir -p /opt/tomcat
- sudo tar -xzf apache-tomcat-10.1.49.tar.gz -C /opt/tomcat --strip-components=1
- sudo chown -R tomcat:tomcat /opt/tomcat
- sudo chmod +x /opt/tomcat/bin/*.sh
-
- echo "Creating Tomcat service..."
- sudo tee /etc/systemd/system/tomcat.service > /dev/null <<'EOF'
- [Unit]
- Description=Apache Tomcat
- After=network.target
-
- [Service]
- Type=forking
- User=tomcat
- Group=tomcat
- Environment="JAVA_HOME=/usr/lib/jvm/java-17-amazon-corretto"
- Environment="CATALINA_HOME=/opt/tomcat"
- Environment="CATALINA_BASE=/opt/tomcat"
- ExecStart=/opt/tomcat/bin/startup.sh
- ExecStop=/opt/tomcat/bin/shutdown.sh
- Restart=on-failure
-
- [Install]

- WantedBy=multi-user.target
- EOF
-
- sudo systemctl daemon-reload
- sudo systemctl enable --now tomcat
-
- echo "Tomcat installation completed."

- Press Esc, type :wq, and press Enter to save and exit.

- Give execution permission: chmod 755 install_tomcat.bash

- Run the script: ./install_tomcat.bash

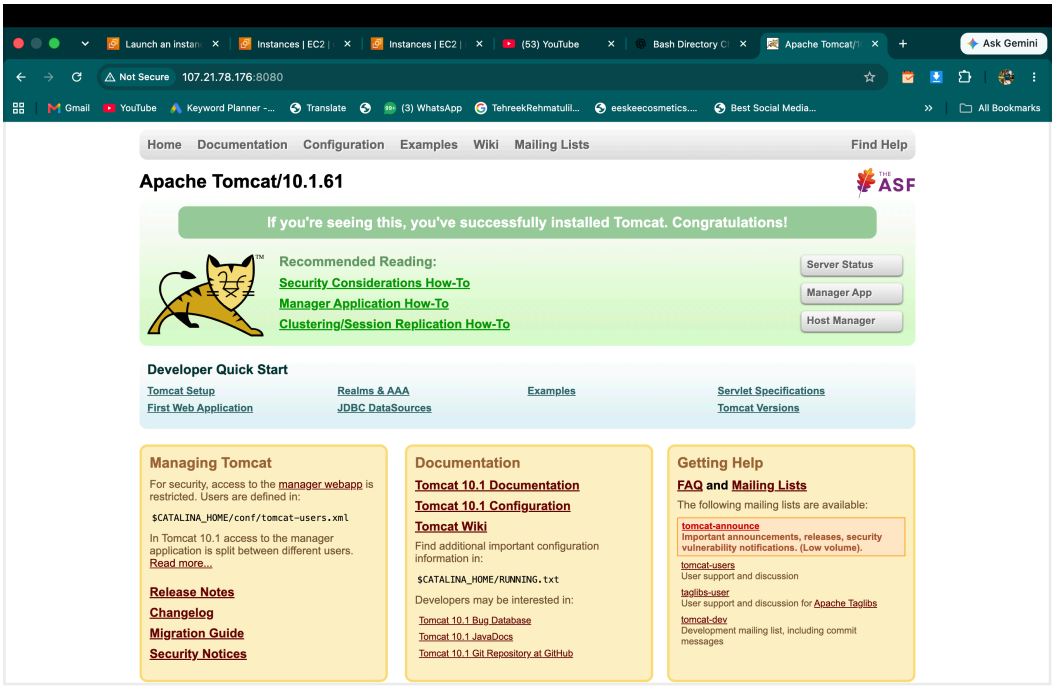
- Verify the Tomcat service: sudo systemctl status tomcat

- Test Tomcat locally: curl -I http://localhost:8080

1.

```
[root@ip-172-31-31-201 scripts]# vi install_tomcat.bash
[root@ip-172-31-31-201 scripts]# ./install_tomcat.bash
Last metadata expiration check: 0:28:55 ago on Wed Oct 21 07:05:35 2026.
Package tar-2:1.34-1.amzn2023.0.4.x86_64 is already installed.
Dependencies resolved.
================================================================================
 Package                    Architecture  Version                    Repository  Size
================================================================================
Installing:
 java-17-amazon-corretto-headless  x86_64  1:17.0.20+13-1.amzn2023.1  amazonlinux  91 M
Installing dependencies:
 cairo                      x86_64  1.18.0-4.amzn2023.0.3      amazonlinux  717 k
 fonts-filesystem           noarch  1:2.0.5-12.amzn2023.0.2    amazonlinux  9.5 k
 google-noto-fonts-common   noarch  20240401-1.amzn2023.0.2    amazonlinux  17 k
 google-noto-sans-vf-fonts  noarch  20240401-1.amzn2023.0.2    amazonlinux  593 k
 graphite2                  x86_64  1.3.14-7.amzn2023.0.3      amazonlinux  97 k
 harfbuzz                   x86_64  7.0.0-2.amzn2023.0.2       amazonlinux  873 k
 javapackages-filesystem    noarch  6.0.0-7.amzn2023.0.6       amazonlinux  12 k
 langpacks-core-font-en     noarch  3.0-21.amzn2023.0.4        amazonlinux  10 k
 libX11                     x86_64  1.8.10-2.amzn2023.0.2      amazonlinux  655 k
 libX11-common              noarch  1.8.10-2.amzn2023.0.2      amazonlinux  144 k
 libXau                     x86_64  1.0.11-6.amzn2023.0.1      amazonlinux  33 k
 libXext                    x86_64  1.3.6-1.amzn2023.0.1       amazonlinux  42 k
 libXrender                 x86_64  0.9.11-6.amzn2023.0.2      amazonlinux  28 k
 libbrotli                  x86_64  1.1.0-1.amzn2023.0.1       amazonlinux  334 k
 libjpeg-turbo              x86_64  2.1.4-2.amzn2023.0.5       amazonlinux  190 k
 libpng                     x86_64  2:1.6.37-10.amzn2023.0.13  amazonlinux  128 k
 libxcb                     x86_64  1.17.0-1.amzn2023.0.1      amazonlinux  235 k
 pixman                     x86_64  0.43.4-1.amzn2023.0.4      amazonlinux  296 k
 xml-common                 noarch  0.6.3-56.amzn2023.0.2      amazonlinux  32 k
Installing weak dependencies:
 alsa-lib                   x86_64  1.2.7.2-1.amzn2023.0.4     amazonlinux  503 k
 dejavu-sans-fonts          noarch  2.37-16.amzn2023.0.2       amazonlinux  1.3 M
 dejavu-sans-mono-fonts     noarch  2.37-16.amzn2023.0.2       amazonlinux  467 k
```





6.Create a Bash script to check if **nginx service** is running or not.

- If not running, the script should start the service.

- vi check_nginx.bash
- Press i to enter Insert mode.

- Write the following code:
  #!/bin/bash
-
- if systemctl is-active --quiet nginx; then
- echo "Nginx service is already running."
- else

- echo "Nginx service is not running. Starting Nginx..."
- sudo systemctl start nginx
-
- if systemctl is-active --quiet nginx; then
- echo "Nginx service started successfully."
- else
- echo "Failed to start Nginx service."
- fi
- fi

- Press Esc, type :wq, and press Enter to save and exit.

- Give execution permission: chmod 755 check_nginx.bash

- Run the script: ./check_nginx.bash

- Verify the service status: sudo systemctl status nginx

```
[root@ip-172-31-31-201 scripts]# vi check_nginx.bash
[root@ip-172-31-31-201 scripts]# cat check_nginx.bash
#!/bin/bash

if systemctl is-active --quiet nginx; then
    echo "Nginx is already running."
else
    echo "Nginx is not running. Starting Nginx..."
    sudo systemctl start nginx

    if systemctl is-active --quiet nginx; then
        echo "Nginx started successfully."
    else
        echo "Failed to start Nginx."
        exit 1
    fi
fi
[root@ip-172-31-31-201 scripts]# chmod 755 check_nginx.bash
[root@ip-172-31-31-201 scripts]# ./check_nginx.bash
Nginx is already running.
[root@ip-172-31-31-201 scripts]#
```

```
[root@ip-172-31-31-201 scripts]# systemctl stop nginx
[root@ip-172-31-31-201 scripts]# ./check_nginx.bash
Nginx is not running. Starting Nginx...
Nginx started successfully.
[root@ip-172-31-31-201 scripts]#
```

7.Create a Bash script to check if a directory is available or not.
If not, then create the directory.
Create the script using vi: vi check_directory.bash

- Press i to enter Insert mode.

- Write the following code:
  #!/bin/bash
-
- DIR="/kamaal/scripts"
-
- if [-d "$DIR"]; then
- echo "Directory already exists."
- else
- mkdir -p "$DIR"
-
- if [-d "$DIR"]; then
- echo "Directory created successfully."
- else
- echo "Failed to create directory."
- fi
- fi

- Press Esc, type :wq, and press Enter to save and exit.

- Give execution permission: chmod 755 check_directory.bash

- Run the script: ./check_directory.bash

- Verify the directory: ls -ld /kamaal/scripts

```
[root@ip-172-31-31-201 scripts]# check-directory.bash
-bash: check-directory.bash: command not found
[root@ip-172-31-31-201 scripts]# vi check-directory.bash
[root@ip-172-31-31-201 scripts]# cat check-directory.bash
#!/bin/bash
```

8.Create a Bash script to delete the last **3 lines** from a file.

- Create the script using vi: vi delete_last_lines.bash
- Press i to enter Insert mode.

- Write the following code:
  #!/bin/bash
-
- FILE="sample.txt"
-
- if [-f "$FILE"]; then
- head -n -3 "$FILE" > temp.txt && mv temp.txt "$FILE"
- echo "Last 3 lines deleted successfully."
- else
- echo "File not found."
- fi

- Press Esc, type :wq, and press Enter to save and exit.

- Create a sample file with some lines: vi sample.txt

- Give execution permission: chmod 755 delete_last_lines.bash

- Run the script: ./delete_last_lines.bash

- Verify the file contents: cat sample.txt

```
[root@ip-172-31-31-201 scripts]# vi kamaal.txt
[root@ip-172-31-31-201 scripts]# cat kamaal.txt
1. A crow felt very thirsty on a hot summer day.
2. He flew around in search of water for a long time.
3. At last, he saw a pitcher with a little water at the bottom.
4. He tried to drink, but his beak could not reach the water.
5. He looked around and saw small pebbles lying on the ground.
6. A smart idea came to his clever little mind.
7. He picked up the pebbles one by one and dropped them into the pitcher.
8. The water level slowly rose up to the top.
9. The happy crow drank the water and flew away.
10. Moral: Where there is a will, there is a way.

[root@ip-172-31-31-201 scripts]# vi delete_last3.bash
```

```
[root@ip-172-31-31-201 scripts]# chmod 755 delete_last3.bash
[root@ip-172-31-31-201 scripts]# ./delete_last3.bash
Last 3 lines deleted successfully.
[root@ip-172-31-31-201 scripts]# cat delete_last3.bash

#!/bin/bash

FILE="kamaal.txt"

if [ -f "$FILE" ]; then
    head -n -3 "$FILE" > temp.txt && mv temp.txt "$FILE"
    echo "Last 3 lines deleted successfully."
else
    echo "File not found."
fi

[root@ip-172-31-31-201 scripts]# cat kamaal.txt
1. A crow felt very thirsty on a hot summer day.
2. He flew around in search of water for a long time.
3. At last, he saw a pitcher with a little water at the bottom.
4. He tried to drink, but his beak could not reach the water.
5. He looked around and saw small pebbles lying on the ground.
6. A smart idea came to his clever little mind.
7. He picked up the pebbles one by one and dropped them into the pitcher.
8. The water level slowly rose up to the top.
[root@ip-172-31-31-201 scripts]# ./delete_last3.bash
Last 3 lines deleted successfully.
[root@ip-172-31-31-201 scripts]# cat kamaal.txt
1. A crow felt very thirsty on a hot summer day.
2. He flew around in search of water for a long time.
3. At last, he saw a pitcher with a little water at the bottom.
4. He tried to drink, but his beak could not reach the water.
5. He looked around and saw small pebbles lying on the ground.
[root@ip-172-31-31-201 scripts]#
```

9.Create a Bash script to monitor **CPU usage**.

- If CPU > 80%, then send an email notification.

- Create the script using vi: vi monitor_cpu.bash
- Press i to enter Insert mode.

- Write the following code:
  #!/bin/bash
-
- THRESHOLD=80
- EMAIL="your-email@example.com"
-
- CPU_IDLE=$(top -bn2 | awk '/Cpu\(s\)/ {idle=$8} END {print idle}')
- CPU_USAGE=$(awk -v idle="$CPU_IDLE" 'BEGIN {printf "%.0f", 100-idle}')
-
- echo "Current CPU usage: ${CPU_USAGE}%"
-
- if ["$CPU_USAGE" -gt "$THRESHOLD"]; then
- MESSAGE="ALERT: CPU usage is ${CPU_USAGE}% on $(hostname) at $(date)."

- echo "$MESSAGE" | mail -s "High CPU Usage Alert" "$EMAIL"
- echo "CPU usage exceeded 80%. Email alert attempted."
- else
- echo "CPU usage is normal."
- fi

- Press Esc, type :wq, and press Enter to save and exit.

- Replace your-email@example.com with your actual email address.

- Give execution permission: chmod 755 monitor_cpu.bash

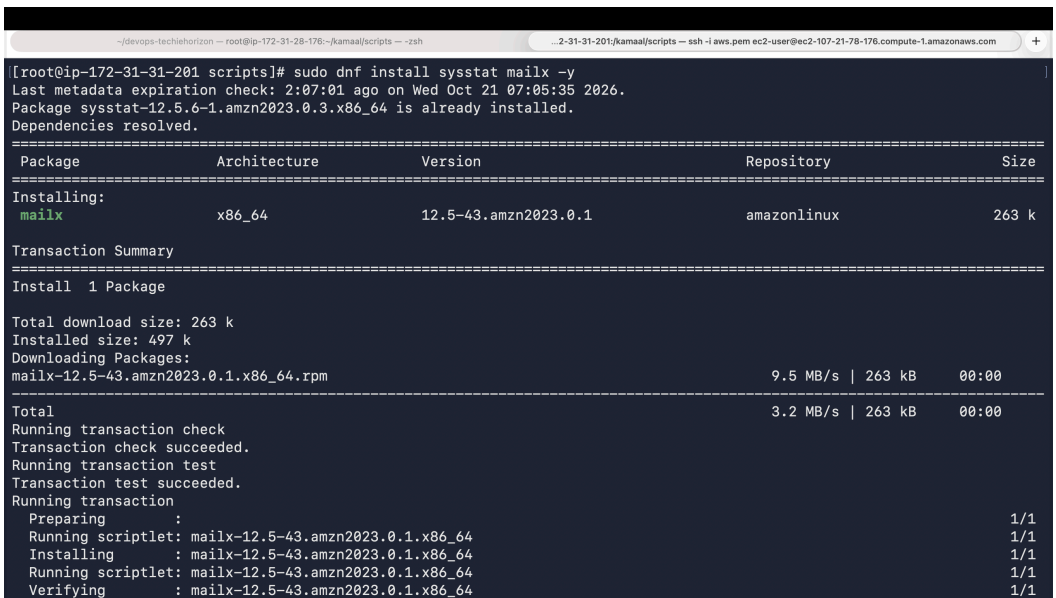
- Run the script: ./monitor_cpu.bash

- Check the current CPU usage output:
  Current CPU usage: 15%
- CPU usage is normal.

- If CPU usage exceeds 80%, the script attempts to send an email alert.

- To install the mail command on Amazon Linux 2023, run:
  sudo dnf install mailx -y

- Important: Installing mailx alone does not guarantee email delivery. Configure an SMTP service or Amazon SES and verify delivery before relying on alerts.



```
Installed:
  mailx-12.5-43.amzn2023.0.1.x86_64

Complete!
[root@ip-172-31-31-201 scripts]# vi cpu-monitor.bash
[root@ip-172-31-31-201 scripts]# cat cpu-monitor.bash
#!/bin/bash
THRESHOLD=80
EMAIL="sharafkamaal@gmail.com"
CPU_IDLE=$(mpstat 1 1 | awk '/Average:/ {print $NF}')
CPU_USAGE=$(awk -v idle="$CPU_IDLE" 'BEGIN {printf "%.0f", 100-idle}')
echo "Current CPU usage: ${CPU_USAGE}%"
if [ "$CPU_USAGE" -gt "$THRESHOLD" ]; then
    echo "ALERT: CPU usage is ${CPU_USAGE}% on $(hostname) at $(date)." \
    | mail -s "High CPU Usage Alert" "$EMAIL"

    echo "CPU usage exceeded 80%. Email alert attempted."
else
    echo "CPU usage is normal."
fi
[root@ip-172-31-31-201 scripts]# chmod 755 cpu-monitor.bash
[root@ip-172-31-31-201 scripts]# ./cpu-monitor.bash
Current CPU usage: 1%
CPU usage is normal.
```

10.Create a Bash script to monitor **Disk Space**.

- If usage > 80%, then send an email notification.

- Create the script using vi: vi monitor_disk.bash
- Press i to enter Insert mode.

- Write the following code:
  #!/bin/bash
-
- THRESHOLD=80
- EMAIL="your-email@example.com"
-
- df -P -x tmpfs -x devtmpfs | awk 'NR > 1 {
- usage = $5
- gsub(/%/, "", usage)
-
- if (usage > 80) {
- printf "%s %s %s\n", $1, $6, usage
- }
- }' | while read -r FILESYSTEM MOUNT USAGE; do
-
- MESSAGE="ALERT: Disk usage is ${USAGE}% on ${MOUNT} (${FILESYSTEM}) on $(hostname) at $(date)."
-
- echo "$MESSAGE" | mail -s "High Disk Usage Alert" "$EMAIL"

-
- echo "$MESSAGE"
- echo "Email alert attempted."
- done
-
- echo "Disk space monitoring completed."

- Press Esc, type :wq, and press Enter to save and exit.

- Replace your-email@example.com with your actual email address.

- Give execution permission: chmod 755 monitor_disk.bash

- Run the script: ./monitor_disk.bash

- Check disk usage manually: df -h

- If any filesystem exceeds 80% usage, the script attempts to send an email notification.

- Install the mail command on Amazon Linux 2023:
  sudo dnf install mailx -y

- Important: Installing mailx alone does not guarantee email delivery. Configure an SMTP service or Amazon SES to send email alerts.

```
[root@ip-172-31-31-201 scripts]# vi disk_monitor.bash
[root@ip-172-31-31-201 scripts]# cat disk_monitor.bash
#!/bin/bash

THRESHOLD=80

df -P -x tmpfs -x devtmpfs |
awk -v limit="$THRESHOLD" '
NR > 1 {
    usage = $5
    gsub(/%/, "", usage)

    printf "Filesystem: %s | Mount: %s | Usage: %s%%\n", $1, $6, usage

    if (usage > limit)
        print "WARNING: Disk usage exceeds 80%!"
    else
        print "Status: Normal"
```

11.Create a Bash script to monitor **Memory Usage**.
    - If usage > 80%, then send an email notification.
- Create the script using vi: vi monitor_memory.bash
- Press i to enter Insert mode.

- Write the following code:
  #!/bin/bash
-
- THRESHOLD=80
- EMAIL="your-email@example.com"
-
- MEMORY_USAGE=$(free | awk '/Mem:/ {
- printf "%.0f", (($2 - $7) / $2) * 100
- }')
-
- echo "Current memory usage: ${MEMORY_USAGE}%"
-
- if ["$MEMORY_USAGE" -gt "$THRESHOLD"]; then
- MESSAGE="ALERT: Memory usage is ${MEMORY_USAGE}% on $(hostname) at $(date)."
-
- echo "$MESSAGE" | mail -s "High Memory Usage Alert" "$EMAIL"
-
- echo "Memory usage exceeded 80%. Email alert attempted."
- else
- echo "Memory usage is normal."
- fi

- Press Esc, type :wq, and press Enter to save and exit.

- Replace your-email@example.com with your actual email address.

- Give execution permission: chmod 755 monitor_memory.bash

- Run the script: ./monitor_memory.bash

- Check memory usage manually: free -h

- If memory usage exceeds 80%, the script attempts to send an email notification.

- Install the mail command on Amazon Linux 2023:
  sudo dnf install mailx -y

- Important: Installing mailx alone does not guarantee email delivery. Configure an SMTP service or Amazon SES to send email alerts.

```
[root@ip-172-31-31-201 ~]# cd /kamaal/scripts
[root@ip-172-31-31-201 scripts]# vi memory_monitor.bash
[root@ip-172-31-31-201 scripts]# cat memory_monitor.bash

#!/bin/bash

THRESHOLD=80
EMAIL="sharafkamaal@gmail.com"

MEMORY_USAGE=$(free | awk '/Mem:/ {
    printf "%.0f", ($3 / $2) * 100
}')

echo "Current memory usage: ${MEMORY_USAGE}%"

if [ "$MEMORY_USAGE" -gt "$THRESHOLD" ]; then
    MESSAGE="WARNING: Memory usage is ${MEMORY_USAGE}% on $(hostname) at $(date)."

    echo "$MESSAGE"
    echo "$MESSAGE" | mail -s "High Memory Usage Alert" "$EMAIL"
else
    echo "Memory usage is normal."
fi

[root@ip-172-31-31-201 scripts]# chmod 755 memory_monitor.bash
[root@ip-172-31-31-201 scripts]# ./memory_monitor.bash
Current memory usage: 34%
Memory usage is normal.
[root@ip-172-31-31-201 scripts]#
```